

강의 자료 · 전 17강 + 특강

나만의 Claude Code 하네스

spec · TDD · CI로 짓는 AI 개발 워크플로우

Red

Green

Refactor

목차

1강	강의 소개	3	9강	TDD 개념 & /test-scenarios	120
2강	Claude와 협업 준비	7	10강	Red · 실패 테스트 작성	132
3강	코드 품질 자동화	20	11강	Green · 최소 구현	142
4강	요구사항 인터뷰	30	12강	Refactor + 보안	157
5강	디자인 시스템 구축	41	13강	이슈 사이클 반복	171
6강	PRD 작성 & 기술 결정(ADR)	52	14강	E2E 개념	177
7강	이슈 분해 & GitHub 등록	68	15강	PR · CI · main 머지	193
8강	기획 워크플로우 자산화	79	16강	/tdd-loop	209
특강	테스트 코드 기초	95	17강	/tdd-auto-loop	218

01

1강

나만의 Claude Code 하네스

spec · TDD · CI로 짓는 AI 개발 워크플로우

강의 워크플로우 한눈에 ①

나만의 AI 하네스 워크플로우 · 15강 코스 · 4단계 · 12스텝

01 환경 셋업 초기 1회

AI 컨텍스트 2강

매 세션마다 프로젝트를 자동 인식

`/init` · `CLAUDE.md` · 컨벤션 주입

첫 Hook 도입 2강

행동을 가로채 자동 적용

`Mermaid` · 자동 다이어그램 · `hook` 패턴 → 자동화

커밋 게이트 3강

매 커밋마다 자동 검증

`husky` · `lint-staged` · `commitlint` · `Conventional Commits`

02 요구사항 · 설계 초기 1회

요구사항 분석 4강

AI 인터뷰로 spec 다듬기

AI 인터뷰 → `spec-fixed` · `Ubiquitous Language` · `Out of Scope`

디자인 & 시스템 아키텍처 5~6강

디자인 시스템 + 기술 결정

디자인 시스템 · `PostToolUse hook` · `PRD` · `ADR 4요소`

이슈 분해 & 스킬화 7~8강

수직 슬라이싱 + 기획 흐름 재사용

수직 슬라이싱 · `Acceptance Criteria` · `/feature-planner` · `gh CLI`

강의 워크플로우 한눈에 ②

TDD 사이클은 이슈마다 반복하고, 완성 단계는 마지막에 한 번 진행합니다.

03 TDD 사이클 이슈 단위 반복

Red · 실패 테스트 9~10강

의도를 코드로 먼저 박기

/test-scenarios · /tdd-red · Vitest · React Testing Library

Green · 최소 구현 11강

YAGNI로 통과시키기

/tdd-green · YAGNI · @ac-verifier · 스킵 vs agent

Refactor · 보안 12강

다듬고 점검

/tdd-refactor · /security-review · tsc --noEmit · npm audit

04 완성 최종 1회

E2E 자동화 14강

사용자 흐름을 자동 검증

/e2e-write · Playwright · Trace Viewer · headed/headless

PR + CI 검증 15강

클린 환경 강제 검증

/create-pr · GitHub Actions · npm ci · 브랜치 보호

main 머지 15강

한 사이클 완성, 다음 사이클로

merge commit · feature → main · docs 정리 · 자산 회수

그래서, AI 하네스가 뭔가요?

업계는 AI 에이전트를 통제하는 모든 장치를 묶어 **하네스(harness)**라고 부릅니다.

Agent = Model + Harness

모델은 이미 충분히 똑똑합니다. **차이는 하네스가 만듭니다.**

여기서 짓는 건 실용적 하네스 한 세트입니다. **가이드 · 검증 · 피드백 루프 · 사람 개입점**, 네 가지 구성요소를 직접 빌드합니다.

→ 학술적 하네스는 메모리 · 플래닝 · 샌드박스 · 다중 에이전트까지 훨씬 넓지만, 여기서는 실전에서 직접 빌드 가능한 실용적 영역에 집중합니다.

02

2강

Claude와 협업 준비

/init과 CLAUDE.md

새 팀에 합류하면 코드부터 읽나요?

"이 팀은 어떻게 일하는가"를 먼저 파악한다.

AI도 마찬가지입니다. 컨텍스트 없이 시키면 **매번 다른 스타일로** 코드를 만듭니다.

지금 여기 있습니다

● 환경 설정 ① · CLAUDE.md · Skill 지금 여기

○ 환경 설정 ② · husky · lint-staged · commitlint

○ 요구사항 & 설계

○ TDD 구현

○ E2E 테스트

○ CI & 정리

이전: 1강에서 워크플로우 동기 확립 · **현재:** CLAUDE.md와 Skill로 Claude 협업 환경 설정

이번 강의 목표

Claude의 기억 모델

세션 단위 컨텍스트와
CLAUDE.md의 역할을
설명할 수 있다

/init으로

CLAUDE.md 생성

초안을 생성하고
프로젝트에 맞게 보강할
수 있다

Skill 설정

SKILL.md 구조와
프로젝트 · 글로벌 설치
위치를 설명할 수 있다

Claude는 세션 간 기억이 없다

LLM은 대화를 저장하지 않습니다. **이것은 버그가 아니라 설계입니다.**

컨텍스트 윈도우

한 번에 처리할 수 있는 텍스트 범위.
세션이 끝나면 사라진다

매 세션 초기화

어제 말한 "named export 쓰세요"를
오늘은 모른다

→ **해결책: CLAUDE.md** — 세션 시작 시 자동으로 로드되는 프로젝트 규칙집

CLAUDE.md = AI 온보딩 문서

README

사람이 읽는 프로젝트 소개

설치 방법, 라이선스

CLAUDE.md

AI가 참조하는 규칙집

코딩 컨벤션, 아키텍처 결정

→ **권장 구성:** 프로젝트 개요 · 파일 구조 · 코딩 컨벤션 · 빌드 & 테스트 명령어 —
200줄 이하 유지

컨벤션 없이 AI가 짜면

```
// export 방식이 파일마다 다르다
// 파일 A
export default function NoteList() {}
// 파일 B
export function NoteEditor() {}
// 파일 C
const NoteItem = () => {}

// 이벤트 핸들러 네이밍이 제각각
handleClick / onClick / onClickHandler
```

→ AI의 문제가 아닙니다. 지침이 없으면 매번 "그럴듯한" 다른 선택을 합니다.

/init — CLAUDE.md 자동 생성

- 1 **핵심 파일 스캔** — package.json, tsconfig.json 등
- 2 **구조 패턴 파악** — 프레임워크, 라이브러리, 아키텍처 분석
- 3 **CLAUDE.md 초안 생성** — 자동 생성된 규칙집. 출발점이지 완성품이 아니다

➔ /init 결과를 그대로 쓰는 것은 바이브 코딩과 같은 실수입니다. 사람이 검토하고 보강해야 합니다.

Mermaid — 다이어그램도 코드처럼

코드는 바뀝니다. 한 달 뒤 다이어그램은 구식이 됩니다. **틀린 문서는 없는 것보다 나쁩니다.**

텍스트로 정의

마크다운처럼 쓰면
다이어그램으로 렌더링

Git으로 관리

diff로 변경 추적, 코드와
함께 버전 관리

AI가 자동 생성

Claude가 코드 분석 →
다이어그램 즉시 생성

→ `/mermaid-diagram` 스킬 하나로 현재 아키텍처를 브라우저에서 확인합니다.

Skill — /명령어를 프로젝트에 등록한다

SKILL.md — 실행 지시서

/mermaid-diagram 입력 → Claude가
.claude/skills/mermaid-
diagram/SKILL.md를 읽고 단계별로 실행

재사용 · 팀 공유

어느 프로젝트에서나 같은 방식으로 호출.
Git으로 팀 전체에 공유 가능

```
.claude/  
└─ skills/  
    └─ mermaid-diagram/  
        └─ SKILL.md ← 실행 지시서
```

Mermaid 스킬 — SKILL.md

```
# Mermaid 아키텍처 다이어그램
```

```
현재 src/ 구조를 분석해 아키텍처 다이어그램을 생성한다.
```

1. src/ 디렉토리 스캔
 - 컴포넌트, 훅, 유틸 파일 목록 수집
 - import/export로 의존성 그래프 구성
2. docs/architecture/index.html 생성
 - CDN: unpkg.com/mermaid (최신)
 - 테마: dark / flowchart TD 형식
3. 브라우저에서 열기
 - macOS: open docs/architecture/index.html
 - Linux: xdg-open docs/architecture/index.html

Skill 설치 위치 — 프로젝트 vs 글로벌

	프로젝트 스킬	글로벌 스킬
경로	<code>.claude/skills/<name>/SKILL.md</code>	<code>~/.claude/skills/<name>/SKILL.md</code>
범위	해당 프로젝트만	모든 프로젝트
공유	Git으로 팀 전체 공유	개인 전용
예시	mermaid-diagram	ppt-maker

➔ `.claude/` 안에 커밋하면 팀 공유, `~/.claude/` 에 두면 혼자 어디서나.

실습

- 1 실습용 밑그림 프로젝트 클론 & Claude Code 연결
- 2 /init으로 CLAUDE.md 생성 후 컨벤션 보강
- 3 현재 프로젝트 코드 분석과 간단한 리팩토링
- 4 Mermaid 아키텍처 스킴 생성

03

3강

코드 품질 자동화

husky · lint-staged · commitlint

AI가 코드를 만든다. 품질은 누가 보장하나?

- Claude가 생성한 코드에 ESLint 에러가 있어도 알림이 없다.
- `npm run lint` 실행을 잊으면 그냥 커밋된다.

사람이 기억해야 할 것은 시스템에 위임한다.

지금 여기 있습니다

- ✓ 환경 설정 ① · CLAUDE.md · Skill
- **환경 설정 ② · husky · lint-staged · commitlint** 지금 여기
- 요구사항 & 설계
- TDD 구현
- E2E · CI

이전: 2강에서 CLAUDE.md 완료 · **다음:** 요구사항 & 설계 단계 진입

이번 강의 목표

Git Hook 원리

pre-commit, commit-msg의 실행 메커니즘과 차단 원리를 설명할 수 있다

도구 설정

husky · lint-staged · commitlint를 설치하고 설정할 수 있다

Conventional Commits

규칙 구조와 목적을 설명하고

CLAUDE.md에 추가할 수 있다

Git Hook — 커밋에 끼어드는 스크립트

```
git commit
└─ .git/hooks/pre-commit 실행
    └─ exit 0 → 커밋 진행
    └─ exit 1 → 커밋 차단
```

pre-commit

커밋 객체 생성 전 · 코드 품질 검사

commit-msg

메시지 기록 전 · 형식 검사

Husky — 훅을 팀이 공유하는 방법

```
# 기존: .git/hooks/ - git 추적 안 됨 → 팀원이 클론하면 훅이 없음  
# Husky: .husky/pre-commit - git 추적됨
```

→ `npm install` 만 하면 팀 전체에 동일한 훅이 자동 적용됩니다.

lint-staged — staged 파일만 빠르게

```
pre-commit → lint-staged: staged된 .ts, .tsx 파일만
├─ ESLint --fix      (자동 수정 가능한 것은 고침)
├─ Prettier --write  (포맷 정리)
├─ 문제 없음        → 커밋 완료
└─ 자동 수정 불가   → 커밋 차단
```

➔ 전체 lint 대신 방금 수정한 파일만 검사 — 빠르고 정확합니다.

Conventional Commits

```
feat(tag): 노트에 태그 추가 기능 구현
fix(tag): 중복 태그 추가 시 에러 수정
test(tag): addTag 유효성 검사 테스트 추가
refactor(tagInput): TagInput 컴포넌트 책임 분리
```

타입	용도
feat	새로운 기능
fix	버그 수정
test	테스트 추가/수정
refactor	동작 변경 없는 구조 개선
chore	빌드, 설정, 패키지 변경

commitlint + 전체 파이프라인

```
git commit -m "feat(tag): 태그 추가"
```

[1] pre-commit → lint-staged 코드 검사

[2] commit-msg → commitlint 메시지 형식 검사

통과 시 커밋 완료

→ commitlint는 형식이 틀리면 커밋을 차단합니다. CLAUDE.md에 규칙을 추가하면 AI도 준수합니다.

실습

husky + lint-staged

설치 및 pre-commit 후 설정

commitlint

설치 및 commit-msg 후 설정

동작 테스트

lint 에러 / 잘못된 메시지 차단 확인

CLAUDE.md 업데이트

Conventional Commits 규칙 추가

04

4강

요구사항 인터뷰

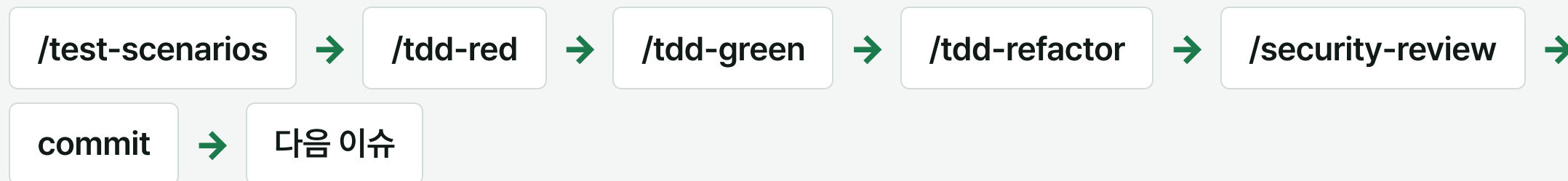
spec-fixed로 결정 확정하기

전체 흐름

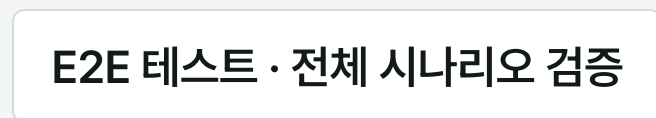
요구사항 & 설계



TDD 구현 — 이슈 단위 반복



모든 이슈 완료 시



"태그 기능 만들어줘"

AI에게 기능을 한 마디로 던지면 AI가 알아서 결정합니다. (제 마음대로...)

별도 테이블? 배열? 최대 개수? 대소문자?

잘못된 결정 위에 쌓이는 코드

발견 시점	상대적 수정 비용
요구사항 단계	1x
설계 단계	5x
구현 단계	10x
테스트 단계	15x
배포 후	100x

→ 요구사항 단계의 1시간 = 배포 후 100시간 절감

지금 여기 있습니다

- ✓ 환경 설정 · CLAUDE.md · Hook · husky · commitlint
- **요구사항 & 설계 · spec-fixed · design.md** 지금 여기
- PRD(ADR) · 이슈 · 자산화
- TDD 구현
- E2E · CI

현재: 요구사항 인터뷰 → spec-fixed · **다음:** 디자인 시스템 구축

이번 강의 포인트

AI 인터뷰

모호한 요청을 구체적
명세로 변환할 수 있다

수정 비용 최소화

요구사항 단계 확정이
비용을 최소화하는
이유를 수치로 설명할 수
있다

spec-original / spec-fixed

분리하여 요구사항
변경의 추적성을 확보할
수 있다

요구사항 엔지니어링

코드

"어떻게"의 영역

요구사항

"무엇을"의 영역

→ AI 시대에는 코드가 빠르게 쌓입니다. → **방향이 더 중요합니다.**

- **기능 요구사항** — 시스템이 무엇을 해야 하는가
- **비기능 요구사항** — 어떤 조건을 만족해야 하는가

AI 인터뷰 — 역할을 뒤집는다

개발자가 묻는 게 아니라 **AI가 묻게 합니다.**

이 기능 정의서를 읽고, 코드베이스를 탐색해서
구현 전에 확인이 필요한 것들을 질문해라.
한 번에 하나씩 질문하고, 추천 방식도 포함해라.

→ AI는 수많은 프로젝트의 실패 패턴을 학습했습니다(나보다 낫다...). 빠진 엣지 케이스를 짚어 냅니다.

3가지 프롬프트 패턴

"한 번에 하나씩"

집중도 유지, 깊이 있는
답변 유도

"추천 방식도 이유와 함께"

판단 근거 제공, 결정 속도
향상

"코드베이스 먼저 탐색"

현실적 질문, 기존 패턴과
충돌 방지

spec-original vs spec-fixed

파일	역할
spec-original.md	원본 기능 정의서
spec-fixed.md	인터뷰 후 확정된 명세

→ **추적성(Traceability)**: "왜 태그 최대 개수가 5개인가?"에 답할 수 있어야 합니다.
변경 이유가 기록되어야 나중에 제약을 바꿔도 되는지 판단할 수 있습니다.

실습 — AI 인터뷰 세션

- 1 **준비** — spec-original.md 준비 → 인터뷰 프롬프트 실행
- 2 **인터뷰 진행** — 한 번에 하나씩 답변 → 결정사항 확정
- 3 **spec-fixed.md 생성** — 인터뷰 결과 + 용어 정의 섹션 포함
- 4 **커밋** — docs(tag): spec-original 및 spec-fixed 작성

05

5강

디자인 시스템 구축

design.md에서 skill · hook까지

지금 여기 있습니다

- ✓ 환경 설정
- ✓ 요구사항 인터뷰
- **디자인 시스템 구축 · design.md · skill · hook** 지금 여기
- PRD(ADR) · 이슈 · 자산화
- TDD 구현
- E2E · CI

현재: 디자인 시스템 구축 → design.md · 시스템 md · skill · hook

이번 강의 포인트

design.md

디자인 이미지를 텍스트
시각 명세로 변환할 수
있다

디자인 시스템 세분화

colors · typography ·
spacing ·
components로
구조화할 수 있다

Skill + PostToolUse Hook

작성된 코드가 디자인
시스템을 자동으로
따르게 만들 수 있다

디자인 이미지로는 부족하다

이미지 한 장으로는 일관된 UI를 만들 수 없습니다. 매번 색 · 간격 · 폰트가 달라집니다.

원인: 단일 진실의 원천(Single Source of Truth)이 없기 때문

- 1 디자인 이미지 (Figma / Stitch / 스크린샷)
- 2 디자인 시스템 생성 (md 파일들) — colors · typography · spacing · components
- 3 Claude Code에 추가 (skill / CLAUDE.md / hook)

→ 이미지를 시스템(문서)으로 옮겨야 AI가 일관되게 따릅니다.

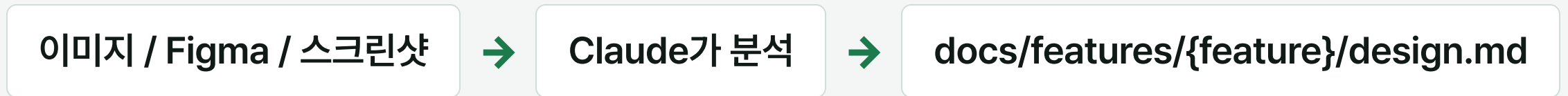
design.md — 시각 명세는 다른 레이어

TDD는 동작을 보장합니다. 하지만 시각 스펙은 테스트로 표현하기 어렵습니다.

- 태그 chip 배경색이 올바른가?
- hover 시 삭제 버튼이 나타나는가?
- 최대 수 도달 시 입력창이 비활성화되는가?

→ → design.md로 관리하고, tdd-green 단계에서 AI가 참조합니다.

design.md 구조



- 컴포넌트별 레이아웃
- Tailwind 클래스
- 상태별 표현 (default / hover / disabled / error)

→ 텍스트 명세가 이미지보다 정확합니다. gray-200인지 gray-300인지 추측할 필요가 없습니다.

Design System Workflow

레퍼런스(Stitch / Figma)에서 디자인 시스템 정보를 가져옵니다.

- 1 Stitch / Figma 레퍼런스
- 2 design.md (시스템 정보 import)
- 3 세분화된 디자인 시스템 md — colors.md · typography.md · spacing.md · components.md
- 4 Skill 생성 — 이 시스템을 따르도록 강제
- 5 PostToolUse Hook — 작성된 코드가 시스템을 지켰는지 검사

→ 디자인 시스템도 결국 문서 + 자동화로 관리합니다.

Claude Code Hook — 자동 실행 스크립트

특정 이벤트 시점에 Claude Code가 실행하는 사용자 정의 스크립트

PreToolUse

도구 실행 전 — 차단 / 검증

PostToolUse

도구 실행 후 — 검사 / 포맷 / 린트

UserPromptSubmit

프롬프트 제출 시 — 컨텍스트 주입

Stop

응답 종료 시 — 마무리 작업

→ `.claude/settings.json` 에 정의 → 사람이 잊어도 시스템이 강제 (3강 git hook과 같은 원리)

Hook 실행 타이밍

- 1 **사용자 입력** → `UserPromptSubmit` — 프롬프트 제출 시 · 컨텍스트 주입 / 전처리
- 2 **Claude 추론 / 판단**
- 3 `PreToolUse` — 툴 호출 시마다 · 차단 / 검증 가능
- 4 **툴 실행** (툴 호출 시 반복)
- 5 `PostToolUse` — 툴 실행 직후 · 검사 / 포맷 / 린트
- 6 **Claude 응답 완료** → `Stop` — 응답 종료 시 · 마무리 작업

PostToolUse Hook 설정 예시

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "node scripts/check-design-system.js"
      }]
    }]
  }
}
```

→ 코드를 작성하면 즉시 디자인 시스템 위반 검사 → 위반 시 AI가 자가 수정

실습

design.md 생성

디자인 레퍼런스 → 텍스트 시각 명세로
변환

디자인 시스템 md 4종

colors · typography · spacing ·
components 세분화

Skill 등록

.claude/skills/design-
system/SKILL.md

PostToolUse Hook

check-design-system.js +
settings.json 등록

06

6강

PRD 작성 & 기술 결정(ADR)

단일 기준 문서 · ADR 4요소 · Out of Scope

지금 여기 있습니다

- ✓ 환경 설정
- ✓ 요구사항 & 설계
- **PRD 작성** **지금 여기**
- 이슈 분해 · GitHub
- TDD 구현
- E2E · CI

요구사항은 완성됐다. 이제 무엇을?

spec-fixed.md가 있습니다.

**그런데 이걸 어떻게 구현의
단일 기준 문서로 만들 것인가?**

이번 강의 목표

PRD 작성

spec과 기술 결정을
하나의 기준 문서로
통합해, 이후 모든 이슈
분해의 출발점으로 쓴다

기술 결정(ADR)

PRD의 "기술 결정"
섹션을 ADR 4요소로
작성해 "왜 이렇게
했는가"를 보존한다

Out of Scope

AI의 범위 초과 구현을
막는 울타리를 세운다

PRD — 모든 이슈의 단일 기준 문서

PRD (Product Requirements Document)

spec-fixed.md (무엇)



기술 결정 (어떤 구조 위에)



하나의 문서

"이 기능에 대해 궁금하면 여기만 보면 된다"

PRD도 진화해 왔다

- 1 **1980~90년대 · 워터폴 시대** — 수백 페이지짜리 "계약 문서". Microsoft · IBM 등이 코딩 전 모든 디테일을 못 박으려 함
- 2 **2000년대 · 애자일의 등장** — "PRD 무용론"과 "가벼운 PRD"로 갈림. 긴 계약서는 변화에 못 따라간다는 비판
- 3 **현재 · 살아 있는 단일 기준 문서** — Notion · Confluence · 리포지토리 안에서 짧고 맥락 있는 형태로 진화

➔ 이 강의는 "가벼운 PRD" 계열입니다. 짧지만 결정의 "왜"까지 담습니다.

PRD의 5가지 구성 요소

개요

기능의 목적과 범위

사용자 스토리

사용자 관점의 흐름

용어 정의

spec-fixed.md 용어를
이어받아 통일

기술 결정

어떤 구조 위에 올릴
것인가

Out of Scope

이번에 하지 않을 것

가장 어려운 건 "기술 결정" 섹션

- 개요 · 사용자 스토리는 spec-fixed.md에서 가져온다.
- Out of Scope은 "하지 않을 것"을 적으면 된다.

기술 결정은? 이제 정해야 한다.

기술 결정은 복리로 작용한다

요구사항이 바뀌면

코드 일부만 수정하면 된다.

기술 결정이 바뀌면

코드 전체를 재구성해야 한다.

→ 앞 이슈의 기술 결정 하나가 뒤 이슈들의 난이도를 전부 바꿉니다.

AI에게 기술 결정을 맡기면 안 되는 이유

팀의 기술 수준

과도한 설계는 오히려
부담이 된다

로드맵

6개월 뒤 추가될 기능을
AI는 모른다

비즈니스 제약

배포 일정, 유지보수
인력을 AI는 모른다

ADR (Architecture Decision Record)

- 1 **2010년 전후 · 문제 상황** — 과도한 문서화 속에서도 "왜 이 기술을 썼지?"를 물으면 아무도 답할 수 없었다
- 2 **Michael Nygard의 제안 (2011)** — "Documenting Architecture Decisions"에서 가볍고 실용적인 형식 제안. 원형은 Status · Context · Decision · Consequences
- 3 **MADR의 확장 (2010년대 후반)** — Nygard 원형에 "대안 비교(Considered Options)"를 더해 지금 널리 쓰는 형태로 진화

→ **핵심 철학:** "결정 자체"에 집중 — 설계 결과물이 아니라 결정을 내린 맥락과 trade-off를 남긴다

ADR의 4요소

코드만 보면 "어떻게"는 알 수 있습니다. **"왜"는 알 수 없습니다.**

Context

이 결정이 필요한 맥락 (당시의 제약 · 요구)

Decision

무엇을 선택했는가

Alternatives

함께 검토한 대안과 거부 이유

Consequences

결정 이후의 결과 (장점 · 단점 · 달라지는 것 모두)

PRD 안에 ADR이 산다

- 별도 ADR 파일을 만들지 않는다.
- PRD의 "기술 결정" 섹션을 ADR 4요소로 쓴다.

→ 파일 하나로 단일 진실의 원천. 결정이 많아지면 그때 분리합니다 (YAGNI).

3가지 안을 같은 기준으로 비교

비교 기준	안 A	안 B	안 C
데이터 구조	Note 안 배열	별도 테이블	배열 + 캐시
API 변경 지점	Note CRUD 확장	Tag API 신설	Note CRUD 확장
상태 관리 변경 지점	Note store 내부	Tag store 신설	Note store + 캐시
즉시 저장 동작	단순	복잡	중간
컴포넌트 구조	기존 재사용	신규 다수	기존 재사용
기존 패턴 일관성	높음	중간	높음
테스트 용이성	높음	중간	높음

➔ 비교해야 좋은 Alternatives 섹션이 나옵니다.

Out of Scope이 가장 중요하다

"태그 색상 커스터마이징은 이번 범위 밖"이라고 명시하지 않으면 **AI가 알아서 구현해 버립니다.**

하지 않을 것의 명시가 할 것의 명시만큼 중요하다

실습

- 1 **PRD 뼈대** — spec-fixed 기반 생성 (기술 결정은 TODO)
- 2 **3안 비교 & 선택** — AI에게 7가지 기준으로 비교 요청
- 3 **ADR 4요소 작성** — Context · Decision · Alternatives · Consequences
- 4 **Out of Scope & 커밋** — 범위 검증 후 커밋

07

7강

이슈 분해 & GitHub 등록

수직 슬라이싱 · Acceptance Criteria · gh CLI

지금 여기 있습니다

- ✓ 섹션 1 · 환경 설정 (1~3강) · CLAUDE.md · husky · hook
- **섹션 2 · 기획 (4~8강) · spec · design · PRD(ADR) · issues · planner**
- 섹션 3 · TDD 구현 (9~13강) · scenarios · Red · Green · Refactor
- 섹션 4 · E2E (14강) · Playwright · PR
- 섹션 5 · CI (15강) · GitHub Actions

지금
여기

PRD 완료 → 이슈 분해 → GitHub Issues 등록

이번 강의 목표

수직 슬라이싱

이슈가 독립적으로
테스트 가능한지 기준을
세워 판단할 수 있다

Acceptance Criteria

Given-When-Then
형식으로 이슈 완료
조건을 작성할 수 있다

gh CLI

분해된 이슈를 GitHub
Issues와 칸반 보드에
자동 등록할 수 있다

PRD가 있다. 이제 뭘 만드나?

PRD를 작성했고, 기술 결정까지 ADR로 담았습니다.

- "전부 한꺼번에"는 TDD로 검증할 수 없다.
- 기능 단위로 쪼개야 사이클을 돌릴 수 있다.

수직 vs 수평 슬라이싱

수평 슬라이싱 ×

- 이슈 #1: API 레이어
- 이슈 #2: Context
- 이슈 #3: UI 컴포넌트
- 이슈 #4: 통합

수직 슬라이싱 ✓

- 이슈 #1: 태그 추가/제거
- 이슈 #2: 태그 카드 표시
- 이슈 #3: 사이드바 필터

각 이슈 완료 시 사용자가 확인 가능

3까지 완료해도

동작하지 않음

수직 슬라이스 판단 기준

"이 이슈만 완료하면 사용자에게
보여 줄 수 있는 동작이 있는가?"

Yes

수직 슬라이스 — TDD 사이클을 돌릴 수
있다

No

다른 이슈와 합치거나 다시 나눈다

TDD의 전제 조건

이슈 하나 = PR 하나 = Red →
Green → Refactor 사이클 하나

- **수평 슬라이싱:** "API 레이어 수정" → 사용자 관점 동작 테스트 불가
- **수직 슬라이싱:** "태그 추가/제거" → 완결된 시나리오 테스트 가능

Acceptance Criteria

Acceptance Criteria

- [] Given 노트 편집 화면에서,
 When 태그 입력 후 Enter를 누르면,
 Then 노트에 태그가 추가되고 화면에 표시된다
- [] Given 이미 존재하는 태그를 입력하고,
 When Enter를 누르면,
 Then 중복 추가되지 않는다

→ AC = 완료 조건 + PR 체크리스트 + 다음 강 /test-scenarios의 씨앗

gh CLI — 터미널에서 GitHub 다루기

이슈 생성

```
gh issue create --title "태그 추가/제거" --body "..."
```

이슈 목록 보기

```
gh issue list
```

PR 생성 (이후 강에서)

```
gh pr create --title "feat(tag): 태그 추가/제거"
```

→ Claude Code가 직접 실행할 수 있습니다. 이슈 목록을 한 번에 등록합니다.

실습

이슈 분해 & 검토

PRD 기반으로 issues.md 생성. 수직
슬라이스 기준 · AC · 의존성 검토

GitHub 등록 & 커밋

gh CLI로 Issues 등록, 칸반 보드 Todo
배치, 커밋

다음 강 예고

8강 — 기획 워크플로우 자산화 & /feature-planner

4~7강을 하나의 워크플로우로 회고하고, 반복되는 기획 과정을 /feature-planner 스킬로 캡슐화합니다.

→ **섹션 2 마무리 · 코스 전체의 중간 정리**

08

8강

기획 워크플로우 자산화 & /feature-planner

4~7강 회고 · Skills · 승인 게이트 · 자체완결형 SKILL.md

지금 여기 있습니다 — 코스의 중간

- ✓ 섹션 1 · 환경 (1~3강)
- 섹션 2 · 기획 · 4 인터뷰 · 5 디자인 · 6 PRD · 7 이슈 · 8 자산화 지금 여기
- 섹션 3 · TDD · 9 scenarios · 10 Red · 11 Green · 12 Refactor · 13 반복
- 섹션 4 · E2E (14강)
- 섹션 5 · PR · CI (15강)

기획은 끝났습니다. 이제 반복을 자산으로 만들고 구현으로 넘어갑니다.

➔ 15강 중 8강 — 코스의 절반 지점. 섹션 2 마무리 ➔ /feature-planner 스킬로 기획 워크플로우를 자산화

이번 강의 목표

회고

4~7강에서 한 일을
하나의 반복 가능한
파이프라인으로 정리

Skills 이해

name · description ·
context: fork · 승인
게이트

/feature-planner 자산화

콜드 세션에서도
동작하는 자체완결형
스킬 생성

잠깐, 우리가 뭘 한 거지?

4강부터 7강까지 **코드 한 줄 쓰지 않고** 한 feature의 기획 파이프라인 전체를 수행했습니다.

→ 이 과정은 태그 기능에만 유효한 게 아닙니다. 검색 · 댓글 · 알림, 어떤 feature에도 동일하게 필요합니다.

4~7강 기획 파이프라인 — 한눈에

4강 · 요구사항 인터뷰

AI가 역질문 ·

Ubiquitous

Language

`spec-fixed.md`

5강 · 디자인 시스템

이미지 → md

시스템 · Hook

`design.md`

6강 · PRD + ADR

3안 비교 · ADR

4요소

`prd.md`

7강 · 이슈 분해

수직 슬라이싱 ·

AC · gh CLI

`issues + GH`

→ 이 4단계가 바로 /feature-planner 스킬이 자산화할 대상입니다.

한 번 해 봤으니, 이제 반복 가능하게 만들 차례다

매번 머릿속에서 재구성하면?

- 어떤 질문은 깜빡하고 빠뜨린다
- 3안 비교의 7가지 기준을 못 외운다
- ADR 4요소 중 Consequences에 단점 쓰는 걸 잊는다
- 수직 슬라이싱 검증을 대충 넘긴다

→ 결과 품질이 들쭉날쭉해집니다.

반복은 자산화한다

훌륭한 개발자는 같은 일을 두 번 하면 자동화를 고민합니다.

코드에서는

함수 / 모듈 / 라이브러리

워크플로우에서는

Claude Code Skill

→ 이번 강에서 기획 파이프라인을 /feature-planner 스킬로 자산화합니다.

Claude Code Skills

```
---  
name: feature-planner  
description: 기능 스펙을 받아서 인터뷰 → PRD(기술 결정 ADR 포함)  
            → 이슈 분해를 순서대로 진행  
context: fork  
---
```

name

/feature-planner로 호출

description

자동 로드 근거

context: fork

서브에이전트로 실행 · 메인
컨텍스트 보호

승인 게이트 — 자동화와 판단의 경계

아키텍처 3안

3가지 안을 제시하고
[GATE]에서 사람이
하나를 고른다

Out of Scope

이번 버전에서 제외할
범위를 [GATE]에서
확정한다

수직 슬라이스

분해된 이슈가 독립 배포
단위인지 [GATE]에서
검토한다

AI는 제안자, 사람은 결정자

스킬은 실행을 자동화한다.
판단은 사람 몫으로 남긴다.

→ 게이트 없이 스킬이 끝까지 달리면, 되돌릴 수 없는 결정이 자동으로 내려집니다.

자체완결형이 핵심

"인터뷰 → PRD → 이슈 분해" 한 줄로 스킬을 만들면? **콜드 세션에서 품질이 무너집니다.**

SKILL.md 본문에 다음을 그대로 박아 넣어야 합니다.

- 질문 템플릿
- 7가지 비교 기준
- ADR 4요소
- 수직 슬라이싱 기준
- Given-When-Then

SKILL.md 구조 미리보기 ①

단계 1: 요구사항 인터뷰

질문 템플릿 (한 번에 하나씩):

1. primary user는?
2. 최소 동작 시나리오 3개?
3. 경계 조건 (에러/빈값/동시성)?
4. 성공 기준은?

[GATE] spec-fixed.md 확인

SKILL.md 구조 미리보기 ②

```
## 단계 2: PRD 작성
```

```
### 아키텍처 3안 비교 (7가지 고정 기준)
```

```
| 기준 | A | B | C |
```

```
| 1. 데이터 구조 | | | |
```

```
| 2. API 레이어 | | | | ... (7개까지)
```

```
[GATE] 안 선택
```

```
### ADR 4요소: Context / Decision / Alternatives / Consequences
```

SKILL.md 구조 미리보기 ③

단계 3: 이슈 분해

판단 기준: "이 이슈만 완료하면 사용자 동작이 있는가?"

AC 형식: Given-When-Then

[GATE] 수직 슬라이스 검토

실습

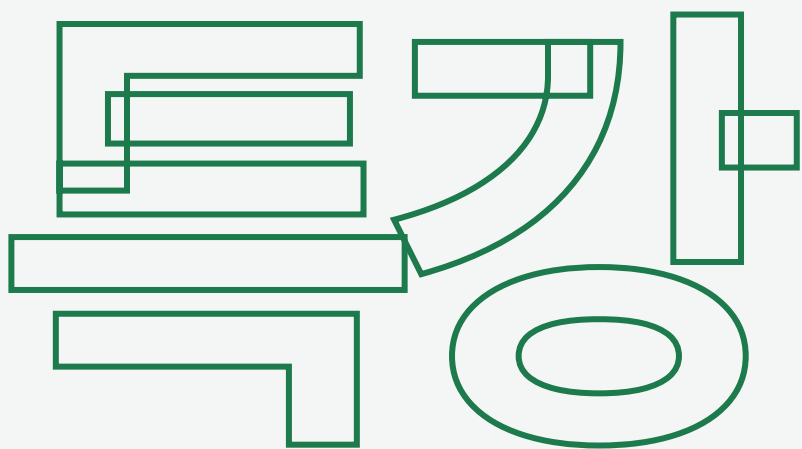
- 1 **레퍼런스 배치** — feature-planning-workflow.md를 프로젝트에 배치
- 2 **스킬 생성** — 레퍼런스 기반으로 SKILL.md 자체완결형 생성 (200줄+)
- 3 **검증 & 시뮬레이션** — 체크리스트 7항목 + 다른 기능("search") 시뮬레이션
- 4 **커밋** — `feat(skills): /feature-planner` 자산화

다음 강 예고

9강 — TDD 개념 & /test-scenarios

섹션 3 진입. 7강에서 등록한 이슈를 가져와 시그니처 확정과 테스트 시나리오 도출을 수행합니다.

→ AC가 곧 테스트 시나리오의 씨앗이 됩니다.



특강

테스트 코드 기초

테스트 코드를 처음 보는 사람을 위해

이번 특강 목표

테스트가 뭔지

자판기 검사 비유로 한 줄에 이해한다

GWT 골격

Given-When-Then으로 자연어처럼
테스트를 읽고 쓴다 (AAA는 참고)

실제 React 예시 2개

버튼 클릭 · 비동기 데이터 — 그대로 옮겨
쓸 수 있다

Matcher · Mock · FIRST

꼭 알아야 할 최소한만

테스트가 뭐냐 — 자판기 비유

자판기를 새로 설치했습니다. 출시 전에 검사합니다.

**"1000원 넣고 콜라 버튼을
누르면 → 콜라가 나와야 한다."**

이게 테스트의 전부입니다. **"이렇게 하면 / 이렇게 돼야 한다"** 한 줄의 약속.

검사 시나리오

구분	내용
주어진 상황	자판기에 콜라 5병이 있고
어떤 행동	1000원 넣고 콜라 버튼을 누르면
기대 결과	콜라 1병이 나오고, 재고는 4병이 된다

→ 대상이 자판기 대신 함수 · 컴포넌트일 뿐, 구조는 똑같습니다.

왜 굳이 코드로 — 수동 vs 자동

자판기 검사를 매번 손으로 한다고 가정해 봅시다.

- 부품 하나 바꿀 때마다 → 다시 1000원 넣고 누르고...
- 100번 검사하려면 → 100번 반복
- 사람은 지친다 · 실수한다 · 빠뜨린다

→ 코드로 한 번 적어 두면 명령 한 줄로 100번이고 1000번이고 1초 안에 다시 검사합니다.

검사 절차를 코드로 적어 둔다

```
test('1000원 넣고 콜라 버튼 → 콜라 나옴', () => {  
  vendingMachine.insert(1000);  
  const drink = vendingMachine.press('cola');  
  expect(drink).toBe('cola');  
});
```

→ 한 번 적어 두면 `npm test` 한 줄로 두고두고 재실행합니다.

테스트 코드의 정체

test(...)

한 가지 약속을 적은 한 묶음

expect(실제값).toBe(기대값)

같으면 통과(초록), 다르면 실패(빨강)

**테스트 코드 = "이렇게 하면 이렇게 돼야
한다"는 약속을 코드로 적어 둔 체크리스트**

가장 작은 예시

```
// 검사 대상
function add(a, b) {
  return a + b;
}

// 검사
test('1 + 2 = 3', () => {
  const result = add(1, 2); // 행동
  expect(result).toBe(3);    // 기대
});
```

GWT — 테스트의 자연어 골격

테스트 한 개는 항상 이 3줄 구조입니다.

Given

어떤 상황이 주어졌을 때

When

무엇을 하면

Then

어떤 결과가 나와야 한다

→ 자연어로 먼저 적고 → 그대로 코드로 옮깁니다.

GWT — 자연어에서 코드로

[Given] 장바구니에 사과가 2개 들어 있고

[When] 사과 1개를 더 담으면

[Then] 총 사과 개수가 3개가 된다

```
test('사과를 더 담으면 개수가 증가한다', () => {  
  const cart = new Cart(['apple', 'apple']); // Given  
  cart.add('apple'); // When  
  expect(cart.count('apple')).toBe(3); // Then  
});
```

참고 — AAA 패턴 (같은 얘기)

코드 쪽에서는 같은 구조를 다른 이름으로도 부릅니다.

GWT	AAA	의미
Given	Arrange	상황 준비
When	Act	동작 실행
Then	Assert	결과 검증

→ GWT로 생각하면서 씁니다. AAA는 같은 골격의 다른 이름일 뿐입니다.

AAA 주석 스타일

```
test('사과를 더 담으면 개수가 증가한다', () => {  
  // Arrange  
  const cart = new Cart(['apple', 'apple']);  
  // Act  
  cart.add('apple');  
  // Assert  
  expect(cart.count('apple')).toBe(3);  
});
```

React 예시 ① — 버튼 클릭

대상 컴포넌트: "버튼을 누르면 카운트가 1 올라간다"

```
function Counter() {  
  const [count, setCount] = useState(0);  
  return (  
    <>  
      <p>현재: {count}</p>  
      <button onClick={() => setCount(count + 1)}>증가</button>  
    </>  
  );  
}
```

→ 이 컴포넌트의 동작을 사용자 관점에서 검증합니다.

React 예시 ① — 테스트 코드

```
test('증가 버튼을 누르면 카운트가 1 올라간다', async () => {  
  render(<Counter />); // Given  
  expect(screen.getByText('현재: 0')).toBeInTheDocument();  
  
  await userEvent.click(screen.getByRole('button', { name: '증가' })); // When  
  
  expect(screen.getByText('현재: 1')).toBeInTheDocument(); // Then  
});
```

→ 내부 `state(count)`가 아니라 **화면에 뭐가 보이는지**로 검증합니다.

React 예시 ② — 비동기 데이터 로딩

대상 컴포넌트: "마운트되면 사용자 이름이 화면에 뜬다"

```
function UserProfile({ userId }) {  
  const [name, setName] = useState<string | null>(null);  
  
  useEffect(() => {  
    fetchUser(userId).then((u) => setName(u.name));  
  }, [userId]);  
  
  return name ? <p>안녕, {name}님</p> : <p>로딩 중...</p>;  
}
```

→ 비동기: 처음엔 "로딩 중", 잠시 후 이름이 뜹니다.

React 예시 ② — 테스트 코드

```
test('마운트되면 사용자 이름이 화면에 뜬다', async () => {  
  vi.spyOn(api, 'fetchUser') // Given: 가짜 응답  
    .mockResolvedValue({ name: '지영' });  
  
  render(<UserProfile userId="u-1" />); // When: 마운트  
  expect(screen.getByText('로딩 중...')).toBeInTheDocument();  
  
  expect( // Then: 잠시 후  
    await screen.findByText('안녕, 지영님') // 기다린다  
  ).toBeInTheDocument();  
});
```

spyOn이 하는 일

**vi.spyOn(api,
'fetchUser')**

실제 함수 호출을
가로챈다

**.mockResolvedValue(
findByText**

가짜 응답으로
바꿔치기한다 — 진짜
fetch는 일어나지 않는다

화면에 나타날 때까지
기다린다 (비동기 검증)

Matcher — 자주 쓰는 것만

```
expect(3).toBe(3); // 정확히 같은가 (값/참조)
expect({ a: 1 }).toEqual({ a: 1 }); // 객체 안쪽까지 같은가
expect([1, 2, 3]).toContain(2); // 배열에 포함되어 있는가
expect('hello world').toMatch(/world/); // 문자열 패턴 매칭
expect(value).toBeNull(); // null인가
expect(isReady).toBe(true); // 참인가
expect(() => divide(1, 0)).toThrow(); // 에러를 던지는가
expect(button).toBeInTheDocument(); // 화면에 보이는가 (RTL)
await expect(fetchUser('x')).rejects.toThrow(); // 비동기 에러 검증
```

→ **toBe vs toEqual** — 숫자 · 문자열은 toBe, 객체 · 배열은 toEqual. 객체에 toBe는 "같은 메모리 주소?"라서 거의 항상 실패합니다.

Mock / Stub — 가짜로 바꿔치기

테스트 중 "진짜를 쓰면 곤란한 것"이 있습니다.

- 실제 API 호출 → 느리고 네트워크 의존
- 결제 · 이메일 → 진짜로 일어나면 안 됨
- 랜덤 · 현재 시간 → 매번 결과가 달라짐

Stub

정해진 값을 돌려준다 — 값을 받아올 때
(API GET, 시간)

Mock

응답 + 호출 검증 — 뭔가 시켰는지 확인
(결제 · 이메일)

Mock / Stub — 코드

```
// Stub - 정해진 응답을 돌려준다
vi.spyOn(api, 'fetchUser')
  .mockResolvedValue({ name: '지영' });

// Mock - 호출까지 검증한다
// checkout: 결제 처리 + 사용자에게 이메일 발송하는 함수
const sendEmail = vi.fn();
checkout({ userId: 'u-1', sendEmail });

expect(sendEmail).toHaveBeenCalledTimes(1);
expect(sendEmail).toHaveBeenCalledWith('u-1@example.com');
```

→ **남용 주의** — 다 가짜로 바꾸면, 통과해도 실제로는 안 되는 코드가 생깁니다.

통과 · 실패가 의미하는 것

✓ 사과를 더 담으면 개수가 증가한다 (12 ms)

← Pass: 약속이 지켜졌다

x 증가 버튼을 누르면 카운트가 1 올라간다
expected '현재: 1' but received '현재: 0'

← Fail: 약속이 깨졌다. 어디?

➔ 한 번도 빨개진 적 없는 테스트는 신뢰할 수 없습니다. 그래서 TDD에서는 일부러 먼저 실패시킵니다 (Red 단계).

FIRST — 좋은 테스트의 5가지 조건

글자	한 줄 설명
F ast (빠르게)	1초 안에 끝나야 자주 돌린다
I ndependent (독립적)	다른 테스트 결과에 영향받지 않는다
R epeatable (반복 가능)	몇 번 돌려도 같은 결과 — 시간/네트워크/랜덤 의존 X
S elf-validating (자가 검증)	사람이 로그를 안 봐도 통과/실패가 자동으로 나온다
T imely (제때)	구현 직전 또는 함께 — 한참 뒤에 몰아 쓰지 않는다

FIRST — Repeatable 위반 예시

× 나쁜 예 — 시간 의존

```
// 오늘 시간에 따라 결과가 달라짐
expect(getGreeting())
  .toBe('Good morning');
// 밤에 돌리면 실패
```

✓ 좋은 예 — 시간 주입

```
// 시간을 인자로 받는다
expect(
  getGreeting(new Date('2026-01-01T09:00'))
).toBe('Good morning');
```

→ 외부 상태(시간 · 네트워크 · 랜덤)에 의존하는 부분은 주입받게 바꿔야 Repeatable합니다.

왜 미리 짜 놓고 자동으로 돌리느냐

회귀 방지

새 기능을 만들다 옛 기능이 안 깨졌나 1초 만에 확인

자신감

고쳐도 되겠다는 허락 — 리팩토링이 무섭지 않다

명세 역할

코드가 어떻게 동작해야 하는지가 테스트에 적혀 있다

AI와 협업의 안전장치

AI가 마음대로 고쳐도 테스트가 깨지면 들킨다

핵심 요약

테스트의 정의

"이렇게 하면 이렇게 돼야 한다"는 약속을 코드로 적은 체크리스트

GWT (Given-When-Then)

코드에서는 AAA

React 검증

사용자가 보는 것
(getByRole /
getByText)으로 검증

비동기

findBy / waitFor로
기다린다

외부 의존

Mock / Stub으로
바꿔치기 — 남용 금지

좋은 테스트

FIRST — 빠르고 ·
독립적이고 · 반복 가능

09

9강

TDD 개념

/test-scenarios 스킬

지금 여기 있습니다

- ✓ 환경 설정
- ✓ 요구사항 & 설계
- **/test-scenarios · 시그니처 확정 · 시나리오 도출** 지금 여기
- /tdd-red → /tdd-green → /tdd-refactor
- E2E · CI

섹션 3 TDD 진입 — /test-scenarios

이번 강의 목표

AI one-shot 문제

TDD가 이를 해결하는
원리를 Red → Green →
Refactor로 설명할 수
있다

시그니처 확정

구현 전에 함수
시그니처를 정하고
경계값 시나리오를
도출할 수 있다

/test-scenarios

스킬을 생성하고
실행하여 AC를 검증
가능한 시나리오로
변환할 수 있다

"이슈 #1 구현해 줘"

AI는 테스트와 코드를 **동시에** 만듭니다.

**테스트가 이미 나온 코드 동작을 뒤따라
확인할 뿐이다 — 버그도 함께 통과한다**

one-shot 구현의 구조적 문제

동시 생성

AI가 테스트와 코드를 한 번에 만든다

뒤따라 확인

이미 나온 코드 동작을
테스트가 "맞다"고 확인할
뿐 — 버그도 함께 통과

회귀 불가

코드가 바뀌면 테스트도
함께 바뀐다 — 안전망
역할 불가

→ "모든 테스트 통과" ≠ "코드가 올바르다"

TDD — 순서를 뒤집는다

테스트 먼저

"어떻게 동작해야 하는가"를 코드로 먼저 표현한다

명세 = 테스트

실패하는 테스트가 구현 명세이자 계약이 된다

역전된 종속

코드가 테스트에 종속 — 테스트가 코드에 종속되지 않는다

→ Kent Beck, 1990년대 정립. 핵심은 "순서"입니다.

Red → Green → Refactor

① Red

실패하는 테스트 작성 —
구현이 없어서 실패해야
한다

② Green

최소한의 구현 — 그
테스트만 통과시킨다.
다른 건 하지 않는다

③ Refactor

구조 개선 — 동작은
바뀌지 않는다. 테스트가
안전망

테스트가 설계를 이끈다

```
// 테스트하기 어려운 코드 - 외부 의존성 직접 호출
function addTag(noteId: string, tag: string) {
  const note = db.query("SELECT * FROM notes WHERE id = ?", noteId);
  // ...
}

// 테스트하기 쉬운 코드 - 의존성 주입
function addTag(note: Note, tag: string): Note {
  // 순수 로직만, DB 접근은 호출자가 처리
}
```

→ 테스트 가능한 코드 = 유지보수하기 좋은 코드

AI와 TDD — 각자의 역할

AI가 잘하는 것

- AC → 시나리오 체계적 도출
- 시나리오 → 테스트 코드 변환
- 테스트를 통과하는 최소 구현

사람이 해야 하는 것

- "무엇을 만들어야 하는가" 결정
- 테스트가 의도를 올바르게 표현하는지 판단
- 리팩토링 방향 결정

시그니처 확정 — API 계약을 먼저 정한다

```
// 시그니처만 (구현 코드 없음)  
addTag(noteId: string, tagName: string): Promise<Note>  
  
// 에러: 태그 5개 초과 시 throw '최대 태그 수를 초과했습니다'  
// 에러: 중복 태그 시 throw '이미 추가된 태그입니다'  
// 에러: 빈 문자열 시 throw '태그 이름을 입력해주세요'
```

➔ 테스트 작성 가능 + API의 임의 변경 방지 + 병렬 작업 가능

경계값 분석 & 시나리오 형식

[정상] addTag should add tag to note when valid tag is provided
[경계] addTag should add tag when note has exactly 4 tags (max-1)
[경계] addTag should accept tag with exactly 20 characters (max length)
[예외] addTag should throw '이미 추가된 태그입니다' when tag already exists
[예외] addTag should throw when note already has 5 tags

→ **off-by-one 버그**: `<` vs `<=` 하나 차이 — 경계에서만 발견됩니다. 이 시나리오가 10강에서 `it(...)` 블록이 됩니다.

실습

- 1 **스킬 생성** — /skill-creator로 /test-scenarios 스킬 생성 및 SKILL.md 검토
- 2 **시그니처 확정** — 이슈 #1 기반 함수 타입 · 에러 케이스 승인
- 3 **시나리오 도출** — 정상 / 경계 / 예외 분류 후 AC 대조 및 최종 승인

10

10강

Red ①

실패 테스트 작성

지금 여기 있습니다

- ✓ 환경 설정
- ✓ 요구사항 & 설계
- ✓ /test-scenarios
- **/tdd-red ① 실패 테스트 작성** 지금 여기
- /tdd-green
- /tdd-refactor · /security-review
- E2E · CI

/tdd-red ① — 시나리오를 실패하는 테스트 코드로 변환

이번 강의 목표

Red 상태 정의

실패 이유가 "구현이 없어서"인지 판별하고 Broken과 구분할 수 있다

Vitest + RTL

사용자 관점 테스트를 구현 세부사항 테스트와 대비하여 작성할 수 있다

/tdd-red 실행

스킬을 생성하고 승인된 시나리오를 실패하는 테스트 코드로 변환할 수 있다

테스트를 먼저 작성했는데 바로 통과한다?

**처음부터 통과하는 테스트는
아무것도 보장하지 않는다.**

Green이 될 때 처음 통과해야, 구현이 의도를 만족시킨다는 증거가 됩니다.

Red 상태란 무엇인가

Red = 구현해야 할 명세가 코드로 표현된 상태

→ "테스트가 한 번도 실패한 적 없다면, 그 테스트가 무언가를 잡을 수 있는지 모른다."

단순히 "테스트가 빨간색"이 아닙니다. 실패 이유가 반드시 **"구현이 없어서"**여야 합니다.

올바른 실패 vs 잘못된 실패

올바른 Red – 구현이 없어서 실패

x should add tag to note when valid tag is provided
Error: not implemented

Broken – 테스트 코드 자체의 문제

x Cannot find module './tagValidation'

→ Broken 상태에서는 "몇 개가 실패했는지"도 알 수 없습니다. 진행 추적이 불가능합니다.

Stub 파일 — 올바른 Red를 만드는 방법

```
// src/utils/tagValidation.ts – stub (구현 없음)
export function addTag(noteId: string, tagName: string): Promise<Note> {
  throw new Error('not implemented');
}

export function removeTag(noteId: string, tagName: string): Promise<Note> {
  throw new Error('not implemented');
}
```

➔ stub 파일 ➔ Vitest가 모든 테스트를 수집 ➔ "12개 중 12개 실패". 이것이 구현의 출발점입니다.

Vitest — 테스트 파일 구조

```
import { describe, it, expect } from 'vitest';
import { addTag } from './tagValidation';

describe('addTag', () => {
  it('should add tag to note when valid tag provided', async () => {
    const result = await addTag('note-1', 'react');
    expect(result.tags).toContain('react');
  });
});
```

→ 9강 시나리오의 `should ...` 형식이 그대로 `it(...)` 설명이 됩니다.

React Testing Library — 사용자 관점

× 구현 세부사항 (지양)

```
expect(component.state.tags)
  .toHaveLength(1)
```

state 구조를 바꾸면 테스트가 깨진다

✓ 사용자 관점 (권장)

```
expect(screen.getByText('react'))
  .toBeInTheDocument()
```

사용자가 보는 것을 테스트한다

getByRole (최우선)

스크린 리더가 인식하는 방식 — 접근성과
테스트를 동시에 확보

getById (최후 수단)

구현에 테스트용 코드를 심는 것 — 꼭 필요한
경우에만 사용

실습

/tdd-red 스킬 생성

skill-creator로 SKILL.md 생성 · "구현 코드 수정 금지" 규칙 확인

SKILL.md 검토

테스트 파일 위치 컨벤션 · should...when 형식 · 즉시 실행 절차 확인

/tdd-red 1 실행

시나리오를 실패 테스트로 변환 · stub 파일 생성 · collect 에러 방지

npm test 확인

"N failed, 0 passed" — Red 상태 완성 · 다음 강 Green 준비 완료



11강

Green ② 최소 구현

YAGNI · 점진적 통과 · /tdd-green

지금 여기 있습니다

- ✓ /test-scenarios
- ✓ /tdd-red ①
- **/tdd-green ② 실패 테스트를 통과시키기** 지금 여기
- /tdd-refactor
- E2E · CI

섹션 3 TDD — /tdd-green. 실패한 테스트만 통과시켜라. 그 이상도, 그 이하도 아니다.

이번 강의 목표

YAGNI 원칙

테스트에 없는 기능은 만들지 않는다

점진적 Green

한 테스트씩 통과시키며 나머지를 깨지 않는다

/tdd-green 스킬

직접 만들고 실행한다. 커버리지 결과를 단서로 읽는다

스킬 vs Agent

AC 검증을 통해 둘의 차이를 구분한다

할 일은 하나뿐

Red 단계에서 실패하는 테스트를 만들었습니다. 이제 남은 일은 그 테스트를 통과시키는 것. 그 이상도, 그 이하도 아닙니다.

"이 테스트만 통과시켜. 다른 건 하지 마."

YAGNI — 지금 필요 없으면 만들지 않는다

You Aren't Gonna Need It.

**미리 만든 기능은
대부분 안 쓰임**

Kent Beck, XP의 핵심
원칙

**AI는 자연스럽게 더
만들려 함**

"이런 경우도 있을 수
있으니까" — 멈춰야 한다

기준은 테스트 목록

테스트가 없으면 지금은
안 만든다

너무 많이 구현하면 생기는 일

검증되지 않은 코드

맞는지 틀린지 아무도
모른다

유지보수 비용 증가

요구사항이 바뀌면 안
쓰는 코드까지 같이
고쳐야 한다

복잡성 증가

진짜 필요한 로직이
불필요한 코드에 묻힌다

Stub에서 Green까지 — 한 칸씩 채우기

핵심 리듬

한 테스트씩, 매번 전체 실행

회귀(regression) 감시

새 테스트는 통과했는데 기존 테스트가 깨졌다면 즉시 롤백

시작 (Red)

Tests: 12 failed, 0 passed

반복 사이클 - 12회

1. 최소 구현 (실패 시 5회까지 재시도)
2. npm test 전체 - 통과 + 회귀 확인
3. issue.md [x] 체크 → 다음 실패 테스트 (N+1)

12 → 11 → 10 → ... → 0

Tests: 12 passed, 0 failed ✓

→ 실패 카운트가 곧 진행도입니다. 12 → 11 → ... → 0

최소 구현이란?

```
// 시나리오: "should add tag to note when valid tag is provided"
export async function addTag(
  noteId: string,
  tagName: string
): Promise<Note> {
  const note = await getNote(noteId);
  note.tags.push(tagName);
  await saveNote(note);
  return note;
}
```

→ 중복 · 최대 개수 검사는 다른 테스트의 영역입니다. 이 테스트가 요구하는 동작만 구현하고, 나머지 테스트는 다음 사이클에서 차례로 통과시킵니다.

테스트 커버리지란?

테스트가 실행한 코드의 비율입니다. 코드에 계측(instrumentation)을 심은 뒤 테스트를 돌려서 어떤 라인을 거쳐 갔는지 추적해 계산합니다.

Function

함수가 호출됐나
(가장 느슨)

Statement

각 문장이 실행됐나
(let, foo() 등)

Line

실행된 라인 (≈
Statement)

Branch

if / else 양쪽 다
(가장 엄격)

→ `@vite/test/coverage-v8` 은 V8 엔진 내장 커버리지를 사용합니다. 별도 빌드 없이 빠릅니다.

Statement vs Branch — 같은 테스트, 다른 점수

```
if (x > 0) doSomething();  
// else 없음 - x ≤ 0일 때는 그냥 skip
```

x = 5로만 테스트한 결과:

지표	점수	이유
Statement	100%	if 문과 doSomething() 둘 다 실행
Branch	50%	true 쪽만 거침 · $x \leq 0$ 분기는 안 거침

→ Branch 100% → Statement 100% (역은 성립하지 않음). Branch가 더 엄격해서 진짜 검증력이 있습니다.

커버리지 — 목표가 아니라 단서

× 잘못된 사용

"80% 달성"이 목표 →
의미 없는 테스트만 양산

✓ 올바른 사용

미커버 라인 = 빠뜨린
시나리오 후보. "이
코드는 왜 실행 안 됐지?"

100%가 아니어도 OK

의도적으로 제외한
경로일 수 있음 (불가능
케이스, 외부 의존)

→ 커버리지는 빠진 부분을 보여 주는 거울이지, 채워야 하는 점수표가 아닙니다.

디자인 스펙도 함께 — 동작과 외형

테스트 + design.md

테스트가 동작을 보장 +
design.md가 외형을
보장

처음부터 함께

컴포넌트 구조가
디자인에 맞게
자연스럽게 잡힘

/tdd-green이 자동 참조

스킬이 design.md를
읽음 — 따로 작업할 필요
없음

→ design.md에 없는 시각적 요소는 추가하지 않습니다. 이것도 YAGNI입니다.

스킬 vs Agent — 언제 어떤 걸 쓰나

	스킬 (Skill)	Agent
언제	매번 같은 절차를 따라야 할 때	매번 다르게 판단하며 탐색해야 할 때
정의	단계별 절차 (1 → 2 → 3)	역할 + 판단 기준 (절차 없음)
위치	<code>.claude/skills/</code>	<code>.claude/agents/</code>
호출	<code>/skill명</code>	<code>@agent명</code>
예	<code>/tdd-red, /tdd-green</code>	<code>@ac-verifier</code>

➔ 절차가 정해지면 스킬, 매번 판단이 필요하면 agent.

실습

- 1 **커버리지 모듈 설치** — @vitest/coverage-v8 설치, 스크립트 등록
- 2 **/tdd-green 스킬 생성** — /skill-creator로 SKILL.md 작성
- 3 **실행** — /tdd-green 1로 한 테스트씩 통과
- 4 **커버리지 확인** — npm run coverage로 미커버 라인 해석
- 5 **AC 검증 agent** — 테스트 통과 = 완료 조건 충족인지 독립 검증

이번 강 정리

YAGNI

테스트에 없는 기능은 안 만든다 — 검증되지 않은 코드는 부채

점진적 Green

한 테스트씩 통과 + 매번 전체 실행 — 회귀 즉시 발견

커버리지 = 단서

미커버 라인은 빠진 시나리오 후보. 100%는 목적이 아님

스킬 vs Agent

절차는 스킬, 판단은 agent — 같은 검증도 둘로 풀 수 있다

→ 다음 — Refactor & Security: 12강 /tdd-refactor · /security-review

12

12강

Refactor + 보안

TDD 사이클의 완성

지금 여기 있습니다

- ✓ /test-scenarios
- ✓ /tdd-red ①
- ✓ /tdd-green ②
- **/tdd-refactor ③ 리팩토링** **지금 여기**
- **/security-review 보안 검증** **지금 여기**
- commit
- E2E · CI

/tdd-refactor ③ → /security-review → commit

이번 강의 목표

리팩토링이 뭔지

"동작은 그대로, 구조만
개선"을 적용할 수 있다

/tdd-refactor 실행

중복 · 네이밍 · 단일 책임
문제를 찾고 고칠 수 있다

/security-review 실행

타입 오류 · 취약점 ·
환경변수 노출을
3등급으로 분류할 수
있다

Green이 끝났는데, 코드는 아직 안 끝났다

11강에서 모든 테스트가 통과했습니다. 그런데 급하게 짠 코드, 중복된 로직, 애매한 이름이 남아 있습니다.

Green = 동작함. Refactor = 좋은 코드가 됨.

리팩토링이란?

→ "외부에서 보이는 동작은 그대로 두면서 내부 구조를 개선하는 것" — Martin Fowler

변경	리팩토링인가?
새 기능 추가	× 리팩토링 아님
버그 수정	× 리팩토링 아님
테스트 전부 통과 + 구조만 개선	✓ 리팩토링임

왜 Green 직후에 하는가

Green 단계의 목표

빠르게 통과 — 임시 코드가 남는다

"나중에"의 함정

그 "나중"은 오지 않는다

안전망 완성

전체 테스트 통과 = 변경해도 즉시
감지된다

지금이 최적

맥락이 살아 있고, 테스트가 보호 중인
지금

리팩토링 5가지 점검 기준

중복 제거

같은 로직이 두 곳 이상
반복되는가

네이밍 명확성

data, temp 대신 의도가
보이는 이름인가

단일 책임

한 함수/컴포넌트가 너무
많은 일을 하는가

불필요한 복잡도

매직 넘버, 깊은 중첩,
과한 추상화는 없는가

CLAUDE.md

불일치

프로젝트 컨벤션과
어긋나지 않는가

한 번에 하나씩

변경 하나 → npm test
└─ PASS → 다음 변경
└─ FAIL → 즉시 롤백 → 다른 접근

→ 여러 변경을 한꺼번에 하면 어느 변경이 문제인지 알 수 없습니다. 롤백이 "조금만 더 고치는 것"보다 빠릅니다.

Vitest는 타입을 검사하지 않는다

명령	검증 대상
<code>npm test</code>	동작 검증 (타입 무시, 빠름)
<code>npx tsc --noEmit</code>	타입 검증 (실행 없음, 전체 파일)

→ Vitest는 esbuild를 씁니다. 타입 정보를 완전히 무시하고 변환합니다. **둘 다 통과해야** 동작도 타입도 안전합니다.

npm audit — 공급망 공격 막기

등급	의미	대응
critical	원격 코드 실행 등	즉시 수정
high	데이터 노출, 권한 상승	빠르게 수정
moderate	조건부 위험	검토 후 결정
low	경미한 문제	대부분 무시 가능

➔ `npm audit fix --force` 사용 금지 — 의존성 호환성이 깨질 수 있습니다.

보안 검증 결과 3등급

즉시 수정 필요

빌드를 막는 타입 오류 ·
critical 취약점 · 민감 키
노출

권장 수정

타입 경고 · moderate
취약점

무시 가능

false positive · 개발
의존성의 low 취약점

오늘 배울 것 정리

리팩토링 정의

동작은 그대로, 구조만 개선 — 테스트가
안전망

한 번에 하나

변경 → 테스트 → 다음 · 실패 시 즉시
롤백

tsc + npm test

타입 검증과 동작 검증을 둘 다 돌려야
안전

보안 3등급

즉시 / 권장 / 무시 — AI가 알아서 고치게
두지 않음

실습 6단계 흐름

페이지	단계
Refactor	1. /tdd-refactor 스킬 생성 → 2. SKILL.md 검토 → 3. /tdd-refactor 실행
Security	4. /security-review 스킬 생성 → 5. /security-review 실행
Ship	6. 커밋 → PR → 머지 → 이슈 클로즈 (<code>gh issue close 1</code>)

→ 한 바퀴 = Refactor → Security → Ship. TDD 사이클 한 바퀴 완성.

실습

/tdd-refactor 스킬

/skill-creator로 생성 · 실행 — 5가지
기준으로 점검

전후 코드 비교

리팩토링 전후 비교 · 테스트 전체 통과
확인

/security-review 스킬

/skill-creator로 생성 · 실행 — tsc,
audit, 환경변수 점검

사이클 마무리

커밋 + `gh issue close 1` — TDD 한
바퀴 완성

13

13강

이슈 사이클 반복 → 기능 완성으로

스킬 한 번 만들면 N번 재사용 — AI 워크플로우의 본질

지금 여기 있습니다

- ✓ 이슈 #1 사이클 완료
- 남은 이슈 #2, #3, ... · 스킬 재사용으로 빠르게 지금 여기
- tag 기능 완성
- 14강 E2E
- 15강 PR · CI

섹션 3 TDD — 사이클 1회 완료 → 반복. 남은 이슈에 동일 사이클을 반복해 tag 기능을 완성합니다.

한 이슈에 대한 풀 사이클

단계	하는 일	유형
/test-scenarios N	시그니처 + 시나리오	skill
/tdd-red N	실패 테스트 작성 + stub	skill
/tdd-green N	최소 구현 (테스트 전체 통과)	skill
@ac-verifier N	AC 충족 독립 검증 (테스트 ≠ AC)	agent
/tdd-refactor N	구조 개선	skill
/security-review N	타입 · 보안 점검	skill
commit → PR	feature/tag-spec 머지 → 이슈 클로즈	

→ 스킬 5개 + agent 1개를 남은 이슈마다 그대로 재사용합니다. 한 번 만들면 N번 호출.

사이클을 CLAUDE.md에 박아 두기

2강의 CLAUDE.md

- 프로젝트 설명만

13강의 CLAUDE.md

- 프로젝트 설명
- + TDD 사이클 6단계
- + 인간 게이트 주의
- + agent vs skill 구분

→ CLAUDE.md는 살아 있는 문서입니다. 발견한 패턴을 그때그때 박아 둡니다.

TDD 사이클 디테일 — 한 이슈를 통과시키는 6단계

① 시나리오 도출

/test-scenarios · 시그니처
확정 · 시나리오 N개 ·
Acceptance Criteria

② Red · 실패 테스트

/tdd-red · Vitest · RTL ·
의도 → 테스트

③ Green · 최소 구현

/tdd-green · YAGNI · 최소
통과 · 회귀 감지

④ AC 검증

@ac-verifier (스킬 X,
AGENT) · .claude/agents/
· 독립 검증

⑤ Refactor · 구조 개선

/tdd-refactor · 5가지 점검
기준 · CLAUDE.md 컨벤션 ·
즉시 롤백

⑥ Security · Commit

/security-review · tsc --
noEmit · npm audit ·
commit

두 개의 피드백 루프와 하나의 결정

FEEDBACK 1 · AC 갭 발견 → Red 복귀

@ac-verifier가 빠뜨린 AC를
찾으면, 실패 테스트를 추가해
사이클을 재시작한다.

FEEDBACK 2 · 리팩토링 깨짐 → 즉시 롤백

테스트가 깨지면 변경 직전
상태로 즉시 복원한다. "조금만
더 고치는 것"보다 빠르다.

DECISION · 스킬 vs agent

절차가 정해진 작업 → 스킬.
탐색적 판단 → agent. AC
검증은 후자.

→ 이슈 단위 반복: Red → Green → Verify → Refactor. 이슈마다 반복하고, 다음 이슈로
넘어갑니다.

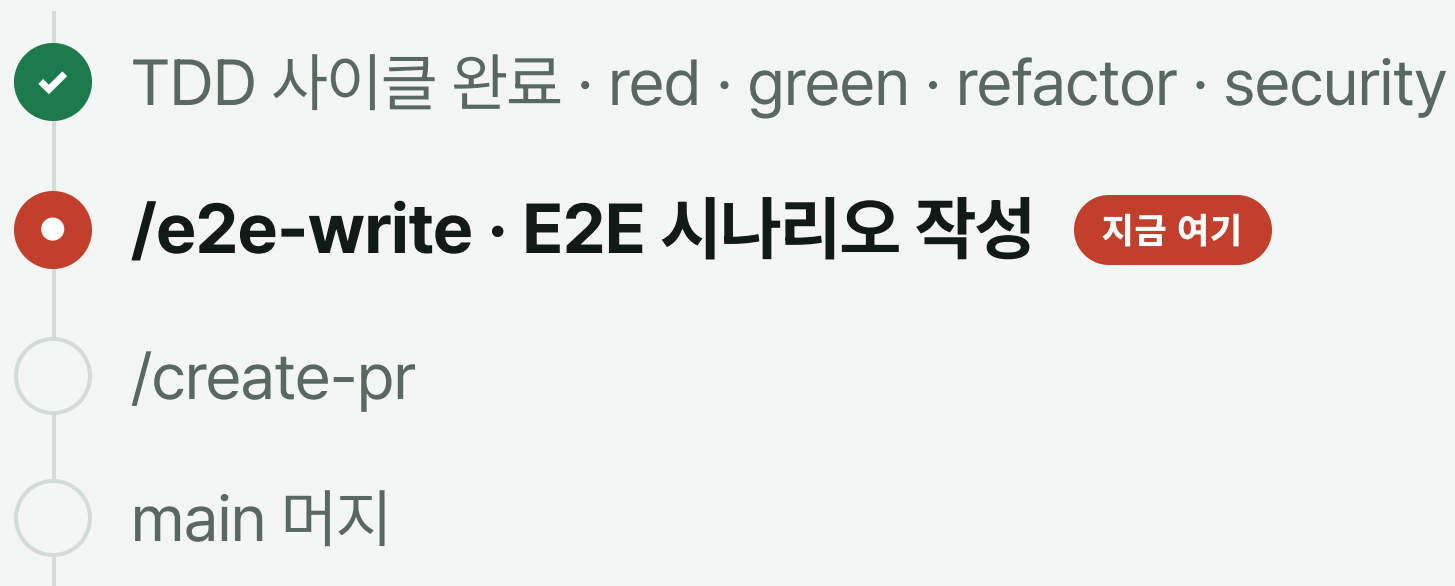
14

14강

E2E 개념

테스트 피라미드 & 시나리오 작성

지금 여기 있습니다



E2E 테스트 — /e2e-write 스킴 생성

이번 강의 목표

테스트 피라미드

E2E의 역할 · "왜 최소한만"인지 설명할 수 있다

/e2e-write 스킬

PRD 사용자 스토리에서 E2E 시나리오 자동 생성 + 실행

셀렉터 우선순위

getByRole → getByText →
getByPlaceholder → getByTestId
순서 적용

headed vs headless

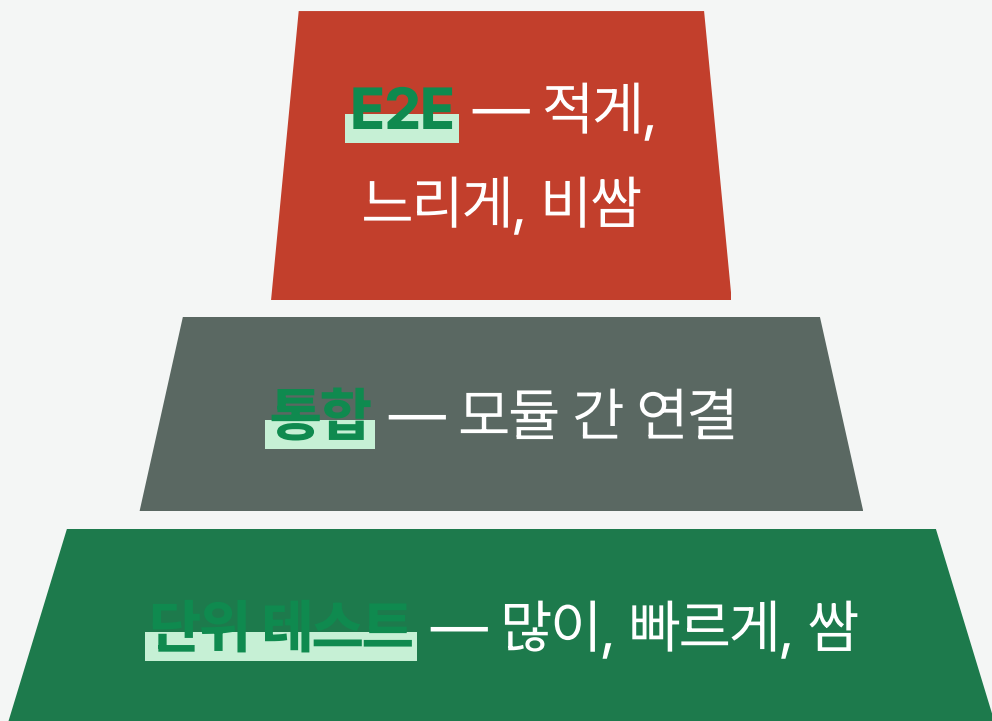
두 실행 모드의 용도 구분

기능은 다 됐는데, 앱은 다 된 건가?

단위 테스트가 모두 초록불이어도, 연결이 끊어져 있으면 사용자는 에러를 만납니다.

함수가 작동한다 ≠ 앱이 작동한다

테스트 피라미드



위로 갈수록 **적게**, 아래로 갈수록 **많이**.

Mike Cohn, *Succeeding with Agile*
(2009)

E2E vs 단위 — 비용과 안정성

	단위 테스트	E2E 테스트
실행 속도	밀리초	초 ~ 분
환경 의존성	없음	브라우저 · 서버 · DB 필요
실패 원인 추적	함수 단위로 즉시	"어딘가 깨졌다"만 알려 줌
Flaky 가능성	거의 없음	네트워크 · 타이밍 불확실

각 레이어가 검증하는 것

단위

addTag() 함수가 올바른 객체를 반환하는가

통합

Context와 API 함수가 연결되어 동작하는가

E2E

"react"를 치고 Enter → chip이 보이는가

E2E 버그를 발견했을 때

- 1 E2E 실패
- 2 어느 레이어에서 깨졌는가? 파악
- 3 해당 단위 테스트에 케이스 추가 (Red)
- 4 수정 (Green) → E2E 재실행 → 통과 확인

→ E2E는 "발견", 단위 테스트가 "방지"합니다.

E2E는 언제 쓰는가

기능의 **모든 이슈가 머지된 후 한 번** 작성합니다.

아직 구현 전인 기능을 E2E로 검증하면 당연히 실패합니다. 그건 버그가 아닙니다.

→ 이후 새 이슈 개발 시: PR 올리기 전마다 전체 E2E 재실행 → 회귀 테스트

Playwright — 왜 이 도구인가

독립성

@playwright/test는 Vitest와 분리 —
기존 테스트 환경 그대로

시맨틱 셀렉터

getByRole, getByText 등 접근성 트리
기반

자동 서버 실행

webServer 설정으로 개발 서버 자동
구동

Trace Viewer

실패 시 스크린샷 · 네트워크 · 콘솔 기록

셀렉터 우선순위 — 접근성 트리 기반

순위	셀렉터	근거
1순위	<code>getByRole</code>	시맨틱 HTML (접근성 지표이기도 함)
2순위	<code>getByText</code>	사용자에게 보이는 텍스트
3순위	<code>getByPlaceholder</code>	input placeholder
4순위	<code>getByTestId</code>	위로 안 될 때만 (최후의 수단)

→ `data-testid` 남용 = 프로덕션 코드에 테스트 전용 속성을 심는 것

E2E 시나리오 설계 4원칙

완결된 사용자 여정

시작부터 끝까지의 흐름을 다룬다

단위 테스트와 중복 금지

"chip이 화면에 보이는가"만 검증

정상 흐름만

예외 / 경계 케이스는 단위 테스트 담당

구체적 행동과 결과

"태그 기능이 잘 동작" × → "Enter →
chip 표시" ✓

테스트 데이터 격리 — Flaky 방지

```
afterEach(async ({ request }) => {  
  for (const id of createdNoteIds) {  
    await request.delete(`${API_URL}/notes/${id}`);  
  }  
  createdNoteIds = [];  
});
```

→ 테스트 독립성 — 어떤 순서로 실행해도 결과가 같아야 합니다.

headed vs headless

```
npx playwright test          # headless (기본, 빠름 · 자동화)  
npx playwright test --headed # 브라우저 창이 직접 보임 (디버깅 · 시연)
```

→ 같은 테스트, 같은 결과 — 모드만 다릅니다.

오늘 배울 것 정리

피라미드

단위는 많이, E2E는 최소한 — 비용 ·
안정성 트레이드오프

시맨틱 셀렉터

getByRole 우선 · testid는 최후의 수단

발견 vs 방지

E2E는 발견 도구 · 단위 테스트가 재발
방지

데이터 격리

afterEach로 정리 · 실행 순서와 무관하게
동일 결과

실습 5단계

- 1 **Playwright 설치** — `npm init playwright@latest`
- 2 **/e2e-write 스킴** — skill-creator로 스킴 생성
- 3 **SKILL.md 검토** — 셀렉터 우선순위 · 데이터 격리 반영 확인
- 4 **/e2e-write tag** — `tests/tag.spec.ts` 생성
- 5 **`npm run test:e2e + show-report`** — 헤드리스 메인 실행 (참고: `--headed`로 브라우저 시각 확인 가능)

15

15강

PR · CI · main 머지

E2E 실행 → PR → CI → main 머지 (코스 마무리)

지금 여기 — 마지막 강

- ✓ spec → PRD → 이슈
- ✓ TDD · red · green · refactor
- ✓ /security-review
- ✓ /e2e-write
- **/create-pr** 지금 여기
- **GitHub Actions CI** 지금 여기
- **브랜치 보호 룰** 지금 여기
- **main 머지 → production** 지금 여기

코스 전체에서 작성한 모든 코드가 production main에 도달하는 시점입니다.

이번 강의 목표

/create-pr 스킴

E2E → push → PR을 한
흐름으로 자동화

Trace Viewer

실패 시 스크린샷 ·
네트워크 · 콘솔로 원인
추적

GitHub Actions CI

클린 환경에서 자동 검증

브랜치 보호 룰

CI 통과 없이 머지 불가능

main 머지

feature/tag-spec →
main 최종 통합

docs 정리

코드가 진실의 원천

/create-pr 스킴의 설계

```
PR 초안 생성 → 개발자 승인
└─ (자동) npm run test:e2e
    └─ FAIL → PR 생성 중단
    └─ PASS → git push → gh pr create
```

"PR 전에 E2E 돌려야지" → 시스템이 강제

E2E FAIL 시 — 스킬이 안내하는 처리 흐름

① Trace Viewer

`npx playwright show-report` → 실패 지점
확인

② 레이어 식별

API · 렌더링 · 로직 — 어디서 깨졌는가?

③ 단위 테스트 추가

해당 단위 테스트에 케이스 추가 (Red)

④ 프로덕션 수정

코드 수정 (Green) → 다시 /create-pr

→ E2E 코드를 고쳐 통과시키는 건 금지입니다. 근본 원인 회피일 뿐. E2E는 발견, 단위 테스트가 방지합니다.

자동화 누적 — 검증 체인

단계	검증 대상	상태
husky + lint-staged	코드 스타일	자동
commitlint	커밋 메시지	자동
/security-review	보안 검토	자동
/create-pr	E2E 테스트	자동
GitHub Actions CI	클린 환경 검증	마지막 빈자리 — 이번 강에서 채움

로컬 검증만으로는 부족하다

"내 머신에서는 됐는데..."

원인: 누락된 의존성 · 환경변수 차이 · 캐시 의존 · 머신 상태 → 내 머신은 클린한 상태가 아닙니다.

→ CI = 격리된 클린 환경에서 "누구의 머신에서도 동작한다"는 객관적 증거

CI 설정 — ci.yml 하나면 끝

```
name: CI
on:
  pull_request: { branches: [main] }
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm ci
      - run: npm run lint
      - run: npx tsc --noEmit
      - run: npm audit --audit-level=moderate
      - run: npm test
```

4단계 — 왜 이 조합인가

단계	잡는 것
<code>npm run lint</code>	코드 스타일 일관성
<code>tsc --noEmit</code>	타입 오류 (Vitest는 타입 검사 안 함)
<code>npm audit</code>	의존성 보안 취약점
<code>npm test</code>	단위 · 통합 · E2E 테스트

→ `npm install` 이 아닌 `npm ci` 사용 → 재현 가능한 빌드

브랜치 보호 룰

GitHub Settings → Branches → Add rule

Require PR before merging

main 직접 push 금지

Require status checks

CI 통과 없이 머지 버튼 비활성화

사람의 의지력 ✕ → 시스템이 강제 ✓

feature → main 최종 머지

```
gh pr create --base main --head feature/tag-spec \  
  --title "feat: 태그 기능 (전체)"  
# CI 자동 실행 → 통과 → 머지 버튼 활성화  
  
gh pr merge --merge    # squash 아님 - 이슈별 히스토리 보존
```

➔ 이게 production에 들어가는 순간입니다. 9~15강 전체가 main으로.

코드가 진실의 원천 — docs 정리

살리는 문서

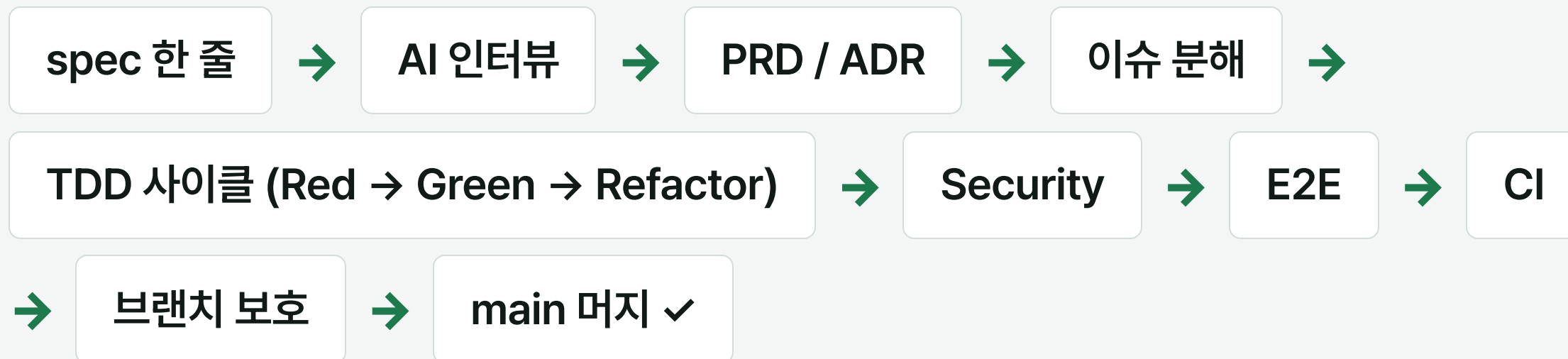
- README · 운영 가이드
- 다이어그램

지우는 문서

- feature 단위 spec, PRD, ADR
- 이슈 분해 작업표

→ **기준:** "6개월 뒤에도 본다?" → 살림. "만들 때만 쓴다?" → 정리.

한 줄 회고



이게 AI 시대의 TDD 워크플로우다.

다음은?

코스는 끝. 다음 기능 개발은 시작.

스킬 · 혹 · 에이전트

만들어 둔 자산을 그대로
재사용

CLAUDE.md

박아 둔 사이클을 AI가
인지한 상태로 시작

사이클 시간

첫 30분 → 두 번째부터
5~10분 (13강 참조)

→ 이 코스 한 바퀴가 가장 큰 투자였고, 이제부터는 그 자산 위에서 굴립니다.

머지 후 — 이슈 한 사이클 (5~10분)

- 1 `gh issue create` → 이슈 등록
- 2 `fix/feature` 브랜치를 `main`에서 분기
- 3 `/test-scenarios` → `/tdd-red` → `/tdd-green`
- 4 `/tdd-refactor` → `/security-review`
- 5 "PR 올려줘" → `/create-pr` 자동 발동
- 6 E2E 자동 → `push` → `PR` → `CI 검증` → 머지

→ 이번 강에서 만든 모든 자산이 그대로 호출되는 자리. `spec` 한 줄에서 `main`까지의 흐름이 자동으로 굴러갑니다.

실습 4단계

1. /create-pr

스킬 생성 + 검토 (description에 자연어 트리거 포함)

2. CI + main 통합 PR

ci.yml을 feature/tag-spec에 추가 → main PR → CI 자기 검증 + 머지

3. 브랜치 보호 룰

다음 PR부터 CI 통과 강제

4. docs 정리

코스 회고 — 코드가 진실의 원천

16

16강

/tdd-loop

TDD 사이클 컨테이너 스킬 — 9~15강 스킬을 하나로 묶는다

지금 여기 있습니다

- ✓ 섹션 0~2 · 환경 · 설계
- ✓ 섹션 3 · TDD 사이클 (9~13강)
- ✓ 섹션 4~5 · E2E · CI · main
- **16강 /tdd-loop · 컨테이너 — 순서 자동화** 지금 여기
- 17강 /tdd-auto-loop
- 18강 워크플로우 유지

섹션 6 — 자동화 진입. 9~15강에서 만든 스킴 · 에이전트를 하나의 호출로 묶습니다.

이번 강의 목표

컨테이너 스킬 개념

기존 스킬을 묶기만 하는 메타 스킬을
설명할 수 있다

/feature-planner 패턴 재인식

같은 구조가 TDD 사이클에도 적용됨을
안다

/tdd-loop SKILL.md 작성

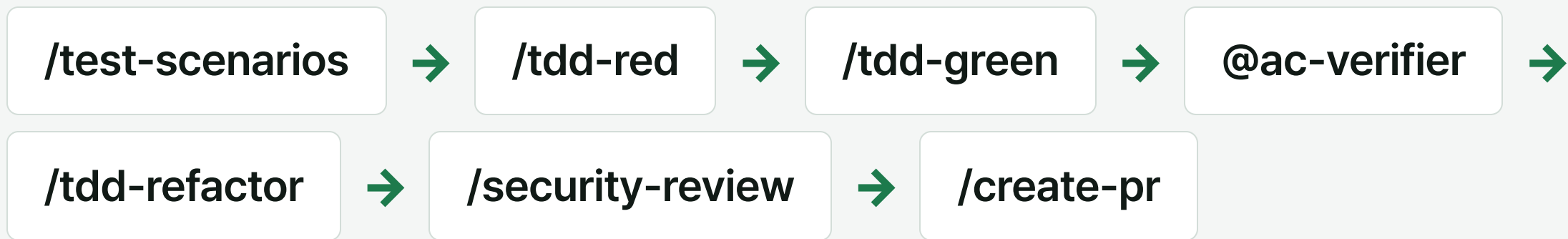
7단계 순서 + base 브랜치 + PR까지 한
흐름으로 묶는다

승인 게이트 유지

컨테이너는 순서만 보장 · 판단은 그대로

한 이슈에 슬래시 7번 — 그 비용

13강까지의 한 사이클을 끝내려면 매번 7번을 누릅니다.



순서 누락 · 명령 잊음 · 컨텍스트 단절 · 반복 노동

이슈 N개 = 7N번

익숙한 패턴 — /feature-planner

8강에서 4 · 5 · 6 · 7강 절차(spec → 인터뷰 → 디자인 → PRD → 이슈 분해)를 하나의 스킬로 묶었습니다.

- 새 기능을 만들 때 한 번 호출
- 내부 단계는 기존 도구가 그대로 실행
- 승인 게이트는 유지 — 컨테이너가 우회하지 않음
- 사람이 누르는 슬래시 명령은 1개

➔ TDD 사이클에도 같은 패턴을 적용합니다.

/tdd-loop — 컨테이너 스킬의 정체

9~15강에서 만든 스킬 · 에이전트를 7단계 순서로 호출하는 메타 스킬

✓ 슬래시 1번 호출 = 7단계 자동 진행

한 번 부르면 끝까지 순서대로 흐른다

✓ 단계 누락 없음 — 순서 강제

빠뜨릴 수 있는 단계가 없다

✓ 단계 실패 위치 명확 보고

어디서 멈췄는지 바로 안다

× 게이트 우회 안 함 — 승인은 그대로

판단은 여전히 사람 몫

7단계 흐름

0. 사전 점검

이슈 AC 확인,
feat/<slug> 분기

1. /test-scenarios

시그니처 · 시나리오
승인 게이트

2. /tdd-red

실패 테스트 작성

3. /tdd-green

최소 구현 (내부
회귀 감시)

4. @ac-verifier

AC 충족 독립 검증

5. /tdd-refactor

리팩토링

6. /security-review

보안 점검

7. /create-pr

commitlint +
Closes #N + PR
생성

브랜치 흐름

```
main
├─ feature/tag-spec      ← 기능 통합 브랜치 (15강에서 main으로)
│  └─ feat/tag-1        ← 이슈 #1 (/tdd-loop이 굴림)
│     └─ PR → feature/tag-spec
│  └─ feat/tag-2        ← 이슈 #2
│     └─ PR → feature/tag-spec
└─ ...                  (이슈 N개 반복)
```

➔ /tdd-loop은 이슈 PR → feature/<spec> 머지까지. main 머지는 15강 패턴을 유지합니다.

정리

/tdd-loop = TDD 사이클의 /feature-planner

순서를 자동화한다. **승인은 그대로.**

17

17강

/tdd-auto-loop

자율 루프 — subagent 오케스트레이션, 사람 개입 없이 PR까지

지금 여기 있습니다

- ✓ 섹션 0~2 · 환경 · 설계
- ✓ 섹션 3 · TDD 사이클
- ✓ 섹션 4~5 · E2E · CI · main
- ✓ 16강 /tdd-loop (컨테이너)
- **17강 /tdd-auto-loop · 자율 — 의사결정 자동화** 지금 여기

섹션 6 — 자동화 심화. 16강 컨테이너의 순서 자동화에서, 17강은 의사결정까지 자동화합니다.

이번 강의 목표

자율 루프 개념

컨테이너와의 차이 — 순서 자동화 →
의사결정 자동화

subagent 오케스트레이션

메인은 결과 JSON만, 코드는 격리된
subagent가

자체 승인 + STOP

하위 스킬의 게이트는 자체 통과, 객관적
실패에서만 STOP

AI가 AI를 검증

Green 직후 ac-verifier로, 사람 게이트
없이 종료

16강 컨테이너의 한계

컨테이너는 명령 7번을 1번으로 줄였지만, **매 스킵의 사용자 승인 게이트는 그대로**입니다.

시그니처 OK? · 시나리오 OK? · 구현 OK? · ...

사람은 여전히 매 단계마다 응답해야 합니다. → 17강이 이 응답까지 자동화합니다.

/tdd-auto-loop — 자율 루프의 정체

각 단계를 subagent로 격리해서 실행하고, **메인은 결과 JSON만 누적**합니다.

- 하위 스킬의 승인 게이트는 자체 통과
- 메인은 코드 본문을 안 본다 — 결과 JSON만
- 객관적 STOP 조건으로만 멈춤 — 추측 금지
- AC 검증은 별도 agent — AI가 AI를 검증

핵심 메커니즘 3가지

컨텍스트 격리

subagent가 단계별로
spawn → 종료 시
컨텍스트 해제. 한 단계의
잡음이 다음으로 새지
않는다.

자체 승인 통과

하위 스킬의 "사용자
승인" 게이트를
subagent가 자체 통과.
사용자에게 묻지 않는다.

객관적 STOP

JSON 결과를 메인이
검사. 기준 미충족 시
깨끗하게 종료. 추측 금지.

7단계 흐름 + STOP 조건

단계	STOP 조건
0. 사전 점검	AC 누락 / uncommitted 변경 / 잘못된 base → STOP
1. 시나리오	시나리오 5개 미만 / 경계값 누락 → STOP
2. Red	잘못된 이유로 실패 (import 에러) → STOP
3. Green (×3)	3회 실패 / 테스트 수정 시도 → STOP
4. AC 검증	ac-passed = false → STOP
5. Refactor	npm test 미통과 → STOP
6. Security + 재스캔	재스캔 후 high 이상 → STOP
7. PR 생성	commitlint 실패 → STOP / 통과 시 PR URL

STOP vs 묻기

× 묻기

"진행할까요?" 출력 후 응답 대기
→ 세션이 매달림, 사용자가 즉시
응답해야 함

✓ STOP

사유 로그 + 이슈 코멘트 + 세션 깔끔히
종료
→ 사용자는 나중에 보고 보강 후 재실행

→ 좋은 이슈는 STOP에 안 걸리고 끝까지 갑니다. **잘 멈추는 것이 자동화의 본체입니다.**

정리

**/tdd-auto-loop = subagent
격리 + 자체 승인 + 객관적 STOP**

의사결정까지 자동화한다. **사람은 PR 리뷰 한 번만.**